BIG DATA ANALYTICS

Tuần 4 — Phần 3: Xử lý dữ liệu theo lô

MapReduce & Apache Spark (bản mở rộng)

Hai mô hình lập trình phân tán làm nền tảng tư duy xử lý Big Data



Biên soạn tham khảo trên nền đề cương BDAN437764 — Big Data Analytics

Khoa Đào tạo Chất lượng cao — Trường Đại học Sư phạm Kỹ thuật TP.HCM

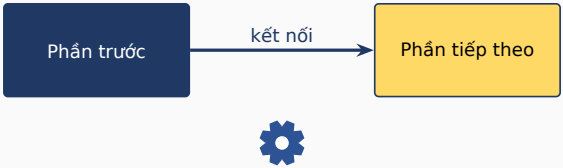
Nội dung buổi học Tuần 4 (bản mở rộng)

- Ôn nhanh: vị trí của Phần 3 trong roadmap 7 phần
- Lý thuyết — MapReduce (chi tiết & ví dụ cụ thể)
  - Mô hình lập trình, luồng xử lý, các thành phần (Combiner, Partitioner)
  - Kiến trúc chạy trên YARN, khả năng chịu lỗi, hạn chế thực tế
- Lý thuyết — Apache Spark (chi tiết & ví dụ cụ thể)
  - Kiến trúc, RDD, Transformation/Action, Shuffle, Persistence
  - DataFrame, Spark SQL, Catalyst Optimizer, Tungsten, AQE
  - Hiệu năng thực tế: Data Skew, đọc Spark UI, case study ETL end-to-end
- Thực hành
  - Lập trình Spark thật với PySpark: RDD, DataFrame, Spark SQL, Windowing
- Tổng kết & chuẩn bị cho Tuần 5

Vì sao nội dung Tuần 4 được mở rộng đáng kể?

- MapReduce & Spark là **công cụ xử lý trung tâm** của toàn bộ khóa học — xuất hiện lại ở hầu hết các phần sau (Streaming, NoSQL/Pipeline, Machine Learning, Kiến trúc end-to-end)
- Hiểu sâu cơ chế bên trong (shuffle, Catalyst, AQE...) giúp viết code Spark hiệu quả thay vì chỉ "dùng được"
- Mỗi khái niệm đều đi kèm **ví dụ cụ thể** (code Java/Python/SQL thực tế) thay vì chỉ mô tả lý thuyết suông

Ôn nhanh: Tuần 3 (Phần 2) chúng ta đã học gì?



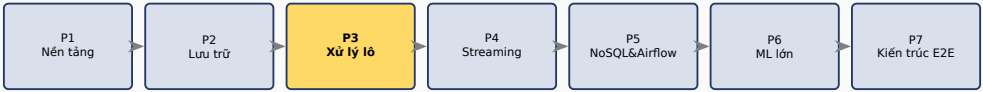
*Phần 2 đã giải quyết bài toán **lưu dữ liệu ở đâu**: HDFS, lưu trữ đám mây, định dạng columnar, kiến trúc Lakehouse (Delta Lake/Iceberg).*

- Dữ liệu giờ đã có nơi lưu trữ đáng tin cậy, có thể mở rộng theo chiều ngang
- Câu hỏi tiếp theo của Phần 3: làm sao **xử lý** khối dữ liệu khổng lồ đó một cách song song, hiệu quả, và với chi phí hợp lý?

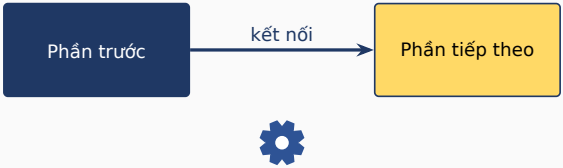
PHẦN 3

MAPREDUCE: MÔ HÌNH LẬP TRÌNH PHÂN TÁN

Nền tảng tư duy xử lý Big Data theo lô



Kết nối từ Phần 2



Dữ liệu đã có nơi lưu trữ đáng tin cậy trên HDFS/Cloud/Lakehouse. Câu hỏi tiếp theo:

- Làm sao viết một chương trình chạy **song song** trên hàng nghìn máy để xử lý dữ liệu đó?

Mục tiêu học tập của Phần 3

- Hiểu mô hình lập trình MapReduce và quy trình phát triển ứng dụng bằng Java, có ví dụ cụ thể
- Hiểu chi tiết vai trò của Combiner, Partitioner, cơ chế Shuffle & Sort, khả năng chịu lỗi
- Hiểu kiến trúc Spark, khái niệm RDD, DataFrame, Spark SQL và các cơ chế tối ưu hóa nội bộ
- Phân tích được ưu điểm/hạn chế của từng mô hình qua các ví dụ và case study thực tế

Bài toán: tính toán song song trên dữ liệu phân tán

- Dữ liệu đã được chia thành các block nằm rải rác trên nhiều DataNode (Phần 2)
- Thay vì kéo hết dữ liệu về một máy để xử lý (tốn băng thông, chậm), ý tưởng cốt lõi là: **đưa chương trình đến nơi dữ liệu đang nằm** (data locality)
- Đây chính là triết lý thiết kế của MapReduce

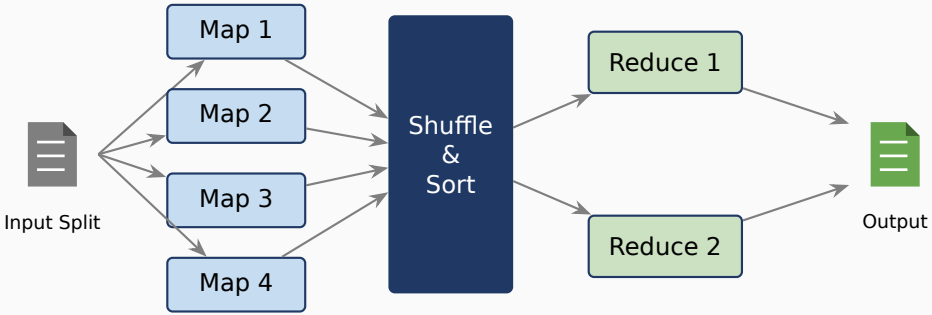
Lịch sử: bài báo MapReduce của Google (2004)

- Dean & Ghemawat, *"MapReduce: Simplified Data Processing on Large Clusters"*, OSDI 2004
- Bối cảnh: Google cần xử lý hàng tỷ trang web để xây dựng chỉ mục tìm kiếm (index) — một máy đơn lẻ không thể đảm đương
- Trước MapReduce, mỗi kỹ sư phải tự viết lại logic phân tán/chịu lỗi cho từng bài toán — MapReduce tách phần đó thành một framework dùng chung

Ý tưởng cốt lõi của MapReduce

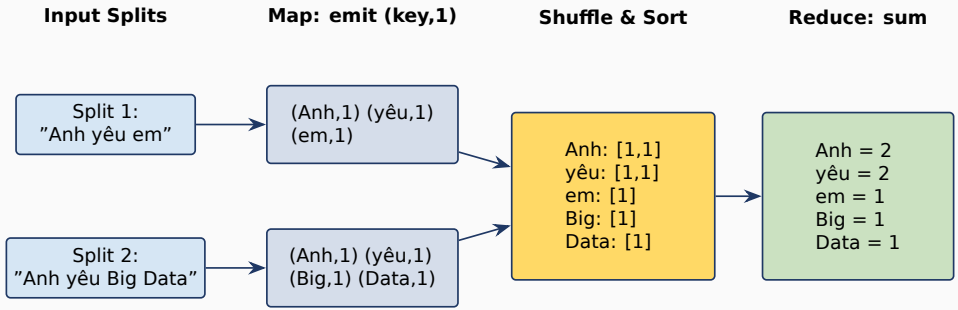
- Lấy cảm hứng từ các hàm `map`/`reduce` trong lập trình hàm (functional programming, ví dụ Lisp)
- Chia bài toán thành hai giai đoạn đơn giản: **Map** (biến đổi từng phần tử độc lập thành cặp key–value) và **Reduce** (tổng hợp các value có cùng key)
- Lập trình viên chỉ cần viết logic Map/Reduce — framework Hadoop tự lo việc phân phối, chịu lỗi, và tổng hợp

Luồng xử lý MapReduce (tổng quan)



Input được chia nhỏ (split) cho nhiều Map task xử lý song song; Shuffle & Sort gom nhóm theo khóa; Reduce tổng hợp kết quả cuối

Ví dụ minh họa cụ thể: WordCount với dữ liệu thật



Hai dòng văn bản được xử lý song song; Shuffle & Sort gom các cặp cùng khóa lại với nhau trước khi Reduce cộng dồn

Ví dụ 1 — WordCount: lớp Mapper (Java)

```java
public class WordCountMapper
    extends Mapper<LongWritable, Text, Text, IntWritable> {

  private final static IntWritable ONE = new IntWritable(1);
  private Text word = new Text();

  @Override
  public void map(LongWritable key, Text value, Context context)
      throws IOException, InterruptedException {
    String line = value.toString();
    for (String token : line.split("\\s+")) {
      word.set(token.toLowerCase());
      context.write(word, ONE);   // emit (word, 1)
    }
  }
}
```

Ví dụ 1 — WordCount: lớp Reducer (Java)

```
public class WordCountReducer
    extends Reducer<Text, IntWritable, Text, IntWritable> {

  @Override
  public void reduce(Text key, Iterable<IntWritable> values,
                     Context context)
      throws IOException, InterruptedException {
    int sum = 0;
    for (IntWritable v : values) {
      sum += v.get();
    }
    context.write(key, new IntWritable(sum));
  }
}
```

Ví dụ 1 — WordCount: lớp Driver / main (Java)

```java
public class WordCountDriver {
  public static void main(String[] args) throws Exception {
    Job job = Job.getInstance(new Configuration(), "word count");
    job.setJarByClass(WordCountDriver.class);
    job.setMapperClass(WordCountMapper.class);
    job.setCombinerClass(WordCountReducer.class);
    job.setReducerClass(WordCountReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
```

Chạy thử Job WordCount bằng Hadoop CLI

```
# Dong goi chuong trinh WordCount thanh file JAR (da bien dich)
hadoop com.sun.tools.javac.Main WordCount*.java -d wc_classes
jar -cvf wordcount.jar -C wc_classes/ .

# Dua du lieu dau vao len HDFS
hdfs dfs -mkdir -p /user/bigdata/input
hdfs dfs -put sample.txt /user/bigdata/input

# Chay Job tren cluster
hadoop jar wordcount.jar WordCountDriver \
  /user/bigdata/input /user/bigdata/output

# Xem ket qua
hdfs dfs -cat /user/bigdata/output/part-r-00000
```

Input Format & Output Format

- **TextInputFormat** (mặc định): mỗi dòng văn bản là một bản ghi, key là vị trí byte, value là nội dung dòng
- **KeyValueTextInputFormat**: tách sẵn key/value theo một ký tự phân cách (ví dụ tab)
- **SequenceFileInputFormat**: định dạng nhị phân của Hadoop, hiệu quả khi dữ liệu đầu ra của job này là đầu vào job khác
- Tương tự có các lớp OutputFormat tương ứng để ghi kết quả đúng định dạng mong muốn

Ví dụ 2 — Bài toán: nhiệt độ trung bình theo trạm khí tượng

- Bộ dữ liệu kinh điển NCDC (National Climatic Data Center): mỗi dòng ghi lại `(mã trạm, năm, nhiệt độ đo được)`
- Yêu cầu: tính nhiệt độ trung bình của **mỗi trạm** qua hàng tỷ bản ghi đo đạc
- Đây là ví dụ minh họa rất rõ điểm khác biệt với WordCount: phép **trung bình** không thể cộng dồn đơn giản như phép đếm

Ví dụ 2 — Mapper: phát ra (mã trạm, nhiệt độ)

```
public class StationTempMapper
    extends Mapper<LongWritable, Text, Text, DoubleWritable> {

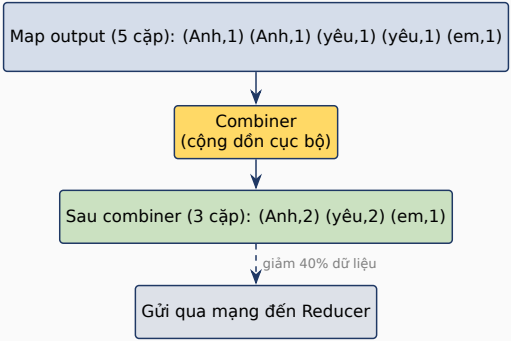
  @Override
  public void map(LongWritable key, Text value, Context context)
      throws IOException, InterruptedException {
    String[] fields = value.toString().split(",");
    String stationId = fields[0];
    double temperature = Double.parseDouble(fields[2]);
    context.write(new Text(stationId), new DoubleWritable(temperature));
  }
}
```

Ví dụ 2 — Reducer: tính trung bình đúng cách

```
public class StationTempReducer
    extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {

  @Override
  public void reduce(Text key, Iterable<DoubleWritable> values,
                     Context context)
      throws IOException, InterruptedException {
    double sum = 0.0;
    long count = 0;
    for (DoubleWritable v : values) {
      sum += v.get();
      count++;
    }
    context.write(key, new DoubleWritable(sum / count));
    // Luu y: KHONG dung Reducer nay lam Combiner truc tiep,
    // vi trung binh cua cac trung binh cuc bo != trung binh tong the
  }
}
```

Combiner: giảm khối lượng dữ liệu truyền qua mạng



Combiner cộng dồn cục bộ ngay tại Map task trước khi gửi qua mạng, giảm đáng kể lưu lượng Shuffle

Combiner — khi nào dùng được, khi nào không?

- Combiner chỉ đúng khi hàm tổng hợp có tính **kết hợp (associative)** và **giao hoán (commutative)**: SUM, MAX, MIN, COUNT đều dùng được
- **Sai lầm thường gặp**: dùng trực tiếp Reducer tính AVERAGE làm Combiner — trung bình của các trung bình cục bộ **không bằng** trung bình tổng thể (ví dụ 2 ở trên)
- Cách khắc phục đúng: Combiner phát ra cặp `(tổng, số lượng)`, Reducer cộng dồn tổng và số lượng rồi mới chia — không tính trung bình sớm

Partitioner — key nào đi Reducer nào?

- Mặc định: **HashPartitioner** — tính `hash(key) mod số_Reducer` để quyết định Reducer nhận key đó
- Custom Partitioner: ví dụ phân loại log truy cập theo tên miền (`.vn`, `.com`, khác) — mỗi nhóm luôn về đúng một Reducer cố định để dễ hậu xử lý
- Partitioner quyết định **mức độ cân bằng tải** giữa các Reducer — chọn sai có thể khiến một Reducer nhận quá nhiều dữ liệu (data skew, sẽ bàn kỹ hơn ở phần Spark)

Shuffle & Sort — điều gì thực sự xảy ra bên trong?

- Mỗi Map task ghi kết quả trung gian ra **đĩa cục bộ**, đã được partition sẵn theo Reducer đích
- Mỗi Reducer kéo (fetch) đúng phần dữ liệu của mình từ tất cả Map task qua mạng — giai đoạn tốn nhiều I/O và băng thông nhất
- Dữ liệu được **sắp xếp (sort)** theo key bằng external merge sort trước khi đưa vào hàm reduce — đây là lý do các giá trị cùng key luôn đến liền kề nhau

Hadoop 1.0 (JobTracker/TaskTracker) vs. Hadoop 2.0 (YARN)

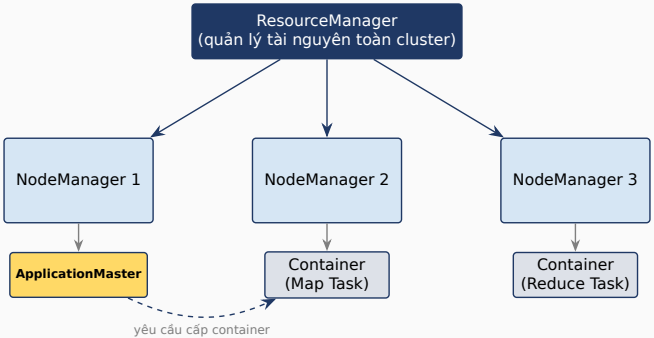
Hadoop 1.0

- JobTracker: vừa lập lịch job vừa quản lý tài nguyên — một điểm nghẽn (bottleneck) duy nhất
- Chỉ chạy được MapReduce, không hỗ trợ framework khác

Hadoop 2.0 (YARN)

- Tách quản lý tài nguyên (ResourceManager) khỏi logic xử lý job (ApplicationMaster)
- Nhiều framework (MapReduce, Spark...) cùng chia sẻ tài nguyên trên một cluster

Chạy MapReduce trên YARN



ResourceManager cấp Container cho ApplicationMaster; ApplicationMaster xin thêm Container để chạy các Map/Reduce Task

Khả năng chịu lỗi (Fault Tolerance)

- Mỗi Task báo cáo tiến độ định kỳ (heartbeat) về ApplicationMaster
- Nếu một Task thất bại (crash, hết bộ nhớ...), nó được **tự động chạy lại** trên một máy khác — không ảnh hưởng các Task còn lại
- Nếu một Task thất bại quá số lần cho phép, toàn bộ Job mới bị coi là thất bại

Speculative Execution — ví dụ cụ thể

- Vấn đề **straggler**: một Task chạy chậm bất thường (ví dụ do máy chủ phần cứng yếu, đĩa sắp hỏng) làm chậm toàn bộ Job dù các Task khác đã xong
- Giải pháp: framework tự động chạy **thêm một bản sao** của Task chậm đó trên một máy khác, đồng thời với bản gốc
- Bản nào hoàn thành trước được giữ lại, bản còn lại bị hủy — đánh đổi thêm tài nguyên để giảm thời gian chờ tổng thể

Ví dụ 3 — Đếm số đơn hàng theo danh mục (ứng dụng E-commerce)

- Nhắc lại ứng dụng Thương mại điện tử từ Tuần 1: log giao dịch hàng triệu dòng mỗi ngày
- Map: đọc mỗi dòng log, phát ra `(danh_mục_sản_phẩm, 1)`
- Combiner + Reducer: cộng dồn giống hệt WordCount — minh chứng WordCount không chỉ là ví dụ "đồ chơi" mà là khuôn mẫu cho rất nhiều bài toán đếm/tổng hợp thực tế

Ưu điểm của MapReduce

- Mô hình lập trình đơn giản: chỉ cần viết hai hàm Map và Reduce
- Tự động chịu lỗi: task thất bại được chạy lại trên máy khác mà không ảnh hưởng toàn bộ job
- Mở rộng tuyến tính: thêm máy vào cluster giúp xử lý nhanh hơn gần như tỉ lệ thuận

Hạn chế của MapReduce

- Kết quả trung gian giữa các giai đoạn Map/Reduce được ghi xuống **đĩa** (không phải RAM) $\Rightarrow$ độ trễ I/O cao
- Không phù hợp với các thuật toán **lặp** (iterative) như nhiều thuật toán học máy — mỗi vòng lặp lại phải đọc/ghi đĩa
- Không hỗ trợ tốt truy vấn tương tác (interactive query) — mỗi job đều có độ trễ khởi động đáng kể

Ví dụ minh họa hạn chế: thuật toán PageRank trên MapReduce

- PageRank cần **lặp nhiều vòng**: điểm hạng của mỗi trang được cập nhật dựa trên điểm hạng các trang liên kết đến nó ở vòng trước
- Trên MapReduce: mỗi vòng lặp là **một Job riêng biệt** — phải đọc toàn bộ đồ thị liên kết từ HDFS, ghi kết quả trở lại HDFS, rồi Job tiếp theo lại đọc lên
- Với đồ thị hàng tỷ trang web và hàng chục vòng lặp để hội tụ, chi phí I/O đĩa lặp lại này cực kỳ tốn kém

Case study thực tế: MapReduce tại Yahoo!/Facebook (2006–2010)

- Yahoo! là nơi khởi xướng dự án Hadoop mã nguồn mở (2006), dùng MapReduce để xây dựng chỉ mục tìm kiếm web quy mô hàng petabyte
- Facebook xây dựng Hive (SQL-on-MapReduce) để đội ngũ phân tích dữ liệu không cần viết Java, phục vụ báo cáo hàng ngày trên log người dùng
- Cả hai case study đều thuộc dạng **batch ETL/báo cáo định kỳ** — đúng thế mạnh của MapReduce, không phải bài toán lặp hay tương tác

Ví dụ 4: Xây dựng Inverted Index (chỉ mục ngược) cho công cụ tìm kiếm

- Đây chính là bài toán **gốc** thúc đẩy Google phát triển MapReduce: xây dựng chỉ mục cho hàng tỷ trang web
- Map: đọc mỗi trang web, phát ra `(từ, tên_trang)` cho mỗi từ xuất hiện trong trang
- Reduce: gom tất cả `tên_trang` ứng với cùng một từ thành một danh sách $\rightarrow$ đây chính là "chỉ mục ngược" dùng để trả lời truy vấn tìm kiếm tức thì

Tổng hợp vai trò các thành phần trong MapReduce

Thành phần	Vai trò
Mapper	Biến đổi mỗi bản ghi đầu vào thành cặp (key, value)
Combiner	Tổng hợp cục bộ tại Map task để giảm dữ liệu truyền qua mạng (tùy chọn)
Partitioner	Quyết định cặp (key, value) nào được gửi đến Reducer nào
Shuffle & Sort	Vận chuyển và sắp xếp dữ liệu theo key trước khi vào Reducer
Reducer	Tổng hợp toàn bộ value ứng với mỗi key, ghi kết quả cuối cùng

Vì sao cần một mô hình mới? Động lực ra đời của Spark

- Ra đời tại AMPLab, UC Berkeley (2009), do Matei Zaharia và cộng sự phát triển, nhằm khắc phục hạn chế I/O đĩa của MapReduce
- Ý tưởng cốt lõi: giữ dữ liệu trung gian trong **bộ nhớ (RAM)** giữa các bước xử lý thay vì ghi xuống đĩa
- Kết quả: nhanh hơn MapReduce hàng chục lần với các workload lặp (machine learning, truy vấn tương tác) — phần tiếp theo sẽ đi sâu vào Spark

Hadoop Ecosystem — MapReduce không đứng một mình

- **Hive**: lớp SQL-on-MapReduce (sau này chạy được trên Spark), giúp viết truy vấn SQL thay vì code Java thuần
- **Pig**: ngôn ngữ kịch bản luồng dữ liệu (dataflow scripting) mức cao hơn MapReduce
- **HBase**: cơ sở dữ liệu NoSQL dạng wide-column xây trên HDFS, sẽ gặp lại ở Phần 5
- Nhiều công cụ trong hệ sinh thái Hadoop dần chuyển sang chạy trên nền Spark thay vì MapReduce thuần vì lý do hiệu năng

Câu hỏi kiểm tra nhanh #1

- Vì sao Combiner tính trung bình trực tiếp lại cho kết quả sai, trong khi Combiner tính tổng (SUM) thì đúng?
- Nếu một Reducer nhận nhiều dữ liệu hơn hẳn các Reducer khác, hiện tượng này gọi là gì và nguyên nhân thường gặp là gì?
- $\Rightarrow$ Hãy thảo luận theo cặp trong 2 phút trước khi chuyển sang phần Spark

Tổng kết: MapReduce

Những điểm mấu chốt:

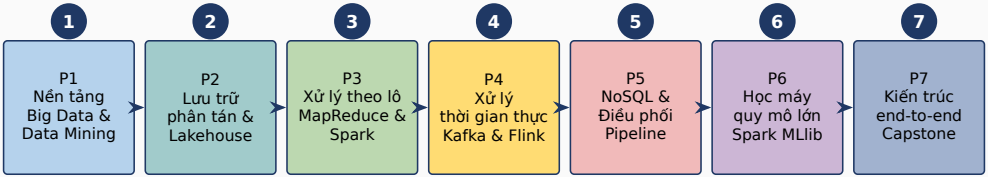
- MapReduce: mô hình Map–Shuffle–Reduce, đơn giản nhưng chậm vì ghi đĩa giữa các bước
- Combiner giảm dữ liệu truyền mạng nhưng chỉ đúng với hàm kết hợp/giao hoán (SUM, MAX...)
- Partitioner quyết định phân bổ tải giữa các Reducer; Shuffle & Sort là giai đoạn tốn kém nhất
- YARN tách quản lý tài nguyên khỏi logic xử lý job, cho phép nhiều framework cùng chia sẻ cluster
- Hạn chế I/O đĩa và không phù hợp thuật toán lặp là động lực trực tiếp dẫn đến sự ra đời của Spark

Câu hỏi ôn tập

1. Trình bày lại luồng WordCount qua 4 giai đoạn: Map, Combine, Shuffle & Sort, Reduce
2. Vì sao PageRank chạy trên MapReduce lại tốn kém hơn nhiều so với chạy trên Spark?
3. Speculative Execution đánh đổi điều gì để lấy lợi ích gì?

APACHE SPARK: KIẾN TRÚC & MÔ HÌNH LẬP TRÌNH

Từ RDD đến DAG, Shuffle, Persistence và các cơ chế bên trong



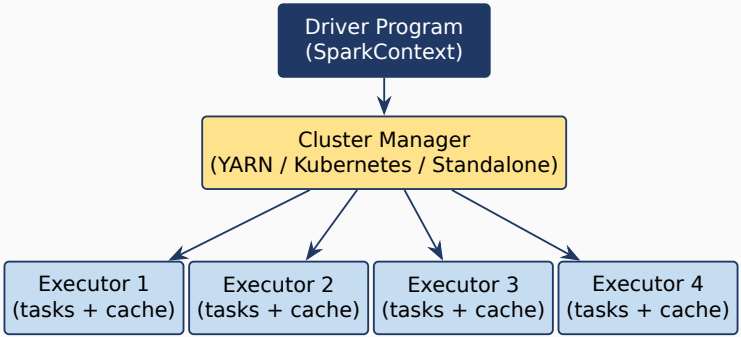
Vì sao cần Spark? (nhắc lại & mở rộng)

- Ra đời tại AMPLab, UC Berkeley (2009), do Matei Zaharia và cộng sự phát triển
- Bài báo nền tảng: Zaharia et al., *"Resilient Distributed Datasets"*, NSDI 2012
- Ý tưởng cốt lõi: giữ dữ liệu trung gian trong **bộ nhớ (RAM)** giữa các bước xử lý thay vì ghi xuống đĩa như MapReduce

Spark trong hệ sinh thái Hadoop

- Spark **không thay thế** HDFS/object storage — nó đọc/ghi dữ liệu từ các hệ thống lưu trữ đó (Phần 2)
- Spark có thể chạy trên nhiều Cluster Manager khác nhau: Standalone, YARN, Kubernetes
- Vì vậy một cluster Hadoop cũ hoàn toàn có thể chạy Spark thay vì (hoặc cùng với) MapReduce mà không cần thay đổi tầng lưu trữ

Kiến trúc Apache Spark



Driver điều phối job; Cluster Manager (YARN/Kubernetes/Standalone) cấp phát tài nguyên; Executor thực thi task và cache dữ liệu

Driver, Executor, Cluster Manager

Driver Program

- Chạy hàm `main()`, tạo SparkSession/SparkContext
- Xây dựng kế hoạch thực thi (DAG) và điều phối các task

Executor

- Tiến trình chạy trên các worker node
- Thực thi task và **cache** dữ liệu trong RAM giữa các bước

So sánh các chế độ Cluster Manager

Chế độ	Đặc điểm	Phù hợp khi
Standalone	Cluster Manager đơn giản đi kèm sẵn Spark, dễ cài đặt	Học tập, cụm nhỏ chuyên dụng cho Spark
YARN	Dùng chung tài nguyên với các job Hadoop/MapReduce khác	Cluster Hadoop có sẵn, nhiều nhóm dùng chung
Kubernetes	Đóng gói Executor thành container, tích hợp hạ tầng cloud-native	Tổ chức đã chuẩn hóa hạ tầng trên Kubernetes

RDD — Resilient Distributed Dataset: 5 đặc tính cốt lõi

- **Danh sách các partition**: RDD được chia thành nhiều phần, mỗi phần xử lý trên một Executor
- **Hàm tính toán** áp dụng cho mỗi partition (compute function)
- **Danh sách các RDD cha** (dependencies) — dùng để tính lại khi mất dữ liệu (lineage)
- **Partitioner** (tùy chọn) cho RDD dạng key–value, và **vị trí ưu tiên** (preferred locations) để tối ưu data locality

Tạo RDD: parallelize() và textFile()

```
from pyspark import SparkContext
sc = SparkContext("local[*]", "Demo")

# Tao RDD tu mot list co san trong bo nho
numbers = sc.parallelize([1, 2, 3, 4, 5], numSlices=2)

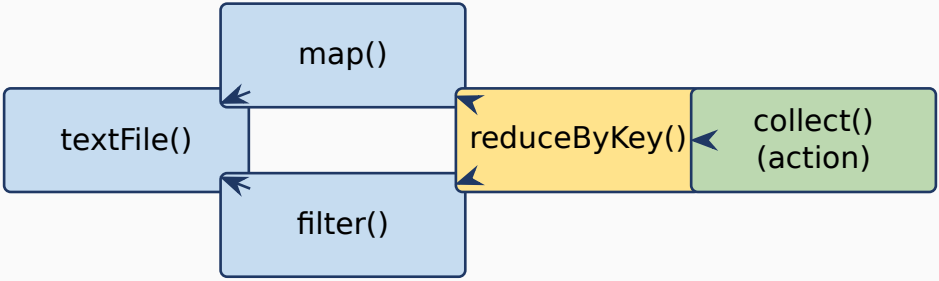
# Tao RDD tu file van ban (co the la file tren HDFS/S3)
lines = sc.textFile("hdfs:///data/logs/2026-08-01.log")

print(numbers.getNumPartitions())   # 2
print(lines.getNumPartitions())     # tuy so block cua file
```

Transformation vs. Action — nguyên lý Lazy Evaluation

- **Transformation** (map, filter, reduceByKey...): *lazy* — chỉ ghi nhận "cách tính", chưa thực thi ngay, trả về một RDD mới
- **Action** (collect, count, saveAsTextFile...): kích hoạt Spark thực sự **thực thi toàn bộ chuỗi** Transformation đã tích lũy
- Lợi ích của lazy evaluation: Spark nhìn thấy **toàn bộ kế hoạch** trước khi chạy, từ đó có thể tối ưu hóa (ví dụ gộp nhiều bước lại)

RDD DAG & Lazy Evaluation



Spark chỉ thực sự thực thi khi gặp một Action — cho phép tối ưu hóa toàn bộ chuỗi Transformation trước khi chạy

Ví dụ Transformations cơ bản: map, filter, flatMap

```
logs = sc.textFile("access.log")

# map: bien doi 1-1
lengths = logs.map(lambda line: len(line))

# filter: giu lai dong thoa dieu kien
errors = logs.filter(lambda line: "ERROR" in line)

# flatMap: 1 dong vao co the sinh ra nhieu phan tu ra
words = logs.flatMap(lambda line: line.split())

print(errors.count())   # Action -> kich hoat thuc thi ca chuoi tren
```

Ví dụ Transformations trên Key-Value RDD

```
pairs = sc.parallelize([
    ("dienthoai", 5), ("laptop", 2), ("dienthoai", 3), ("tivi", 1)
])

# reduceByKey: cong don cuc bo truoc khi shuffle (nhu Combiner)
tong = pairs.reduceByKey(lambda a, b: a + b)
print(tong.collect())
# [('dienthoai', 8), ('laptop', 2), ('tivi', 1)]

# groupByKey: gom nhom TRUOC, khong cong don cuc bo
nhom = pairs.groupByKey().mapValues(list)
print(nhom.collect())
```

reduceByKey vs. groupByKey — vì sao hiệu năng khác nhau?

reduceByKey

- Cộng dồn **cục bộ** trên mỗi Executor trước khi shuffle (giống Combiner của MapReduce)
- Dữ liệu truyền qua mạng **ít hơn nhiều**

groupByKey

- Gửi **toàn bộ** giá trị qua mạng rồi mới gom nhóm
- Có thể gây tràn bộ nhớ nếu một key có quá nhiều giá trị (data skew)

Ví dụ: aggregateByKey và combineByKey

```
# Tinh (tong, so_luong) cho moi khoa de sau nay tinh trung binh dung cach
data = sc.parallelize([("A", 10), ("A", 20), ("B", 5)])

result = data.aggregateByKey(
    (0, 0),                                  # gia tri khoi tao (sum, count)
    lambda acc, v: (acc[0] + v, acc[1] + 1),   # ket hop trong 1 partition
    lambda a, b: (a[0] + b[0], a[1] + b[1])    # ket hop giua cac partition
)
avg = result.mapValues(lambda sc_: sc_[0] / sc_[1])
print(avg.collect())   # [('A', 15.0), ('B', 5.0)]
```

Ví dụ: join và cogroup giữa hai RDD

```
donhang = sc.parallelize([(1, "iPhone"), (2, "Laptop")])
khachhang = sc.parallelize([(1, "An"), (2, "Binh")])

# join: ket hop theo khoa, giong INNER JOIN
ket_hop = donhang.join(khachhang)
print(ket_hop.collect())
# [(1, ('iPhone', 'An')), (2, ('Laptop', 'Binh'))]

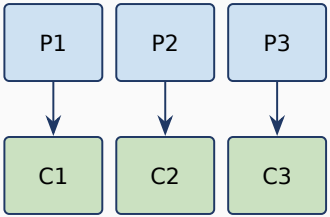
# cogroup: gom nhom gia tri tu ca hai RDD theo khoa, chua join
nhom = donhang.cogroup(khachhang)
```

Danh mục Actions phổ biến

Action	Kết quả trả về
collect()	Toàn bộ dữ liệu về Driver (cẩn thận với RDD lớn)
count()	Số lượng phần tử
first() / take(n)	Phần tử đầu tiên / n phần tử đầu
reduce(f) / fold(zero, f)	Tổng hợp toàn bộ RDD thành một giá trị
foreach(f)	Áp dụng hàm f cho mỗi phần tử (không trả về Driver)
saveAsTextFile(path)	Ghi kết quả ra hệ thống file phân tán

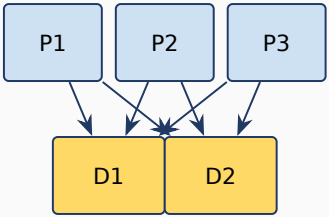
Narrow vs. Wide Dependency

Narrow (map, filter)



Không cần shuffle

Wide (groupByKey, join)



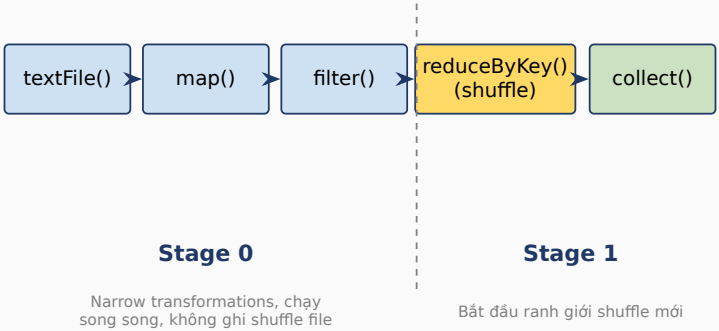
Cần shuffle (all-to-all)

Narrow: mỗi partition con chỉ phụ thuộc một partition cha (không cần shuffle); Wide: phụ thuộc nhiều partition cha (cần shuffle)

Vì sao phân biệt Narrow/Wide Dependency quan trọng?

- **Narrow** (map, filter, union): có thể thực thi **song song hoàn toàn** trên từng partition, không cần trao đổi dữ liệu giữa các Executor
- **Wide** (groupByKey, reduceByKey, join, distinct): cần **shuffle** — trao đổi dữ liệu qua mạng giữa các Executor
- Khi một partition bị mất, Narrow dependency chỉ cần tính lại đúng partition cha tương ứng; Wide dependency có thể cần tính lại nhiều partition cha hơn

DAG được chia thành Stage theo ranh giới Shuffle



Mỗi khi gặp một Wide dependency (ví dụ reduceByKey), Spark kết thúc Stage hiện tại và bắt đầu Stage mới

Lineage & khả năng chịu lỗi của RDD

- RDD **không cần nhân bản (replicate)** dữ liệu như HDFS để chịu lỗi
- Thay vào đó, Spark ghi nhớ **lineage** — chuỗi các Transformation đã tạo ra RDD đó
- Nếu một partition bị mất (Executor chết), Spark chỉ cần **tính lại đúng phần bị mất** dựa trên lineage, không cần tính lại toàn bộ

Storage Level khi cache()/persist()

Storage Level	Ý nghĩa
MEMORY_ONLY	Lưu trong RAM dạng object Java/Python (mặc định của cache())
MEMORY_AND_DISK	Ưu tiên RAM, tràn ra đĩa nếu không đủ bộ nhớ
MEMORY_ONLY_SER	Lưu trong RAM dạng serialized — tiết kiệm bộ nhớ hơn, tốn CPU giải nén
DISK_ONLY	Chỉ lưu trên đĩa cục bộ của Executor

Ví dụ dùng cache() để tránh tính toán lại

```
from pyspark import StorageLevel

logs = sc.textFile("access.log")
errors = logs.filter(lambda l: "ERROR" in l).cache()   # hoac .persist(...)

# Ca hai dong ben duoi deu dung lai ket qua errors da cache,
# khong doc va filter lai tu dau
print("So loi:", errors.count())
errors.saveAsTextFile("output/errors_only")
```

Đo lường tác động của cache() bằng thời gian chạy

```
import time

logs = sc.textFile("access.log")
errors = logs.filter(lambda l: "ERROR" in l)   # CHUA cache

t0 = time.time()
errors.count()
print("Lan 1 (doc + filter lai):", time.time() - t0, "giay")

errors = errors.cache()
errors.count()   # lan nay moi thuc su nap vao cache

t0 = time.time()
errors.count()
print("Lan 2 (da cache):", time.time() - t0, "giay")
# Thuong thay lan 2 nhanh hon ro ret vi khong doc/filter lai tu dau
```

Broadcast Variables

- Vấn đề: khi cần dùng một biến/bảng **nhỏ** (ví dụ bảng tỷ giá ngoại tệ) trong hàm map trên mọi Executor, gửi lại nhiều lần rất lãng phí
- **Broadcast Variable**: gửi một bản sao **chỉ một lần** đến mỗi Executor, dùng lại cho mọi task trên Executor đó
- Ứng dụng phổ biến nhất: **broadcast join** — join một bảng lớn với một bảng nhỏ mà không cần shuffle bảng lớn

Ví dụ Broadcast Variable

```
ty_gia = sc.broadcast({"USD": 25400, "EUR": 27600, "JPY": 170})

don_hang = sc.parallelize([("USD", 100), ("EUR", 50), ("JPY", 2000)])

quy_doi = don_hang.map(
    lambda x: (x[0], x[1] * ty_gia.value[x[0]])   # dung ty_gia.value tren moi Executor
)
print(quy_doi.collect())
```

Accumulators

- Biến **chỉ được cộng dồn** (write-only từ Executor, đọc được ở Driver) — dùng để đếm/theo dõi trong lúc xử lý phân tán
- Ứng dụng phổ biến: đếm số bản ghi lỗi/không hợp lệ trong lúc parse dữ liệu, mà không cần một Action riêng chỉ để đếm
- Lưu ý: Accumulator chỉ đảm bảo chính xác trong **Action**, có thể bị cộng lại nếu Transformation phải tính lại do lỗi

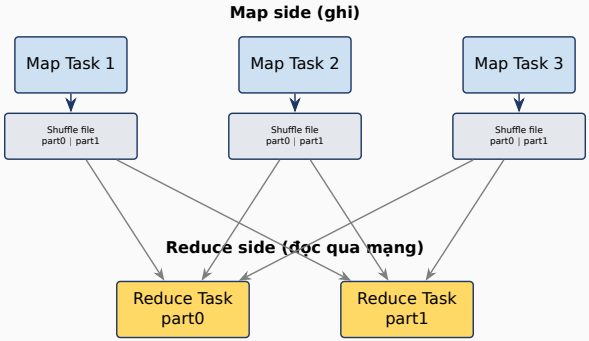
Ví dụ Accumulator: đếm bản ghi lỗi khi parse JSON

```
loi_parse = sc.accumulator(0)

def parse_an_toan(dong):
    global loi_parse
    try:
        return json.loads(dong)
    except Exception:
        loi_parse.add(1)
        return None

ban_ghi = sc.textFile("events.jsonl").map(parse_an_toan).filter(lambda x: x)
print("So ban ghi hop le:", ban_ghi.count())
print("So ban ghi loi:", loi_parse.value)
```

Cơ chế Shuffle nội bộ trong Spark



Map task ghi shuffle file đã phân vùng ra đĩa cục bộ; Reduce task kéo (fetch) đúng phần của mình từ mọi Map task qua mạng

Partitioning: repartition() vs. coalesce()

- **repartition(n)**: shuffle lại toàn bộ dữ liệu để chia thành đúng n partition — dùng khi cần **tăng** số partition hoặc cân bằng lại tải
- **coalesce(n)**: gộp các partition hiện có lại, **tránh shuffle toàn bộ** khi chỉ cần **giảm** số partition
- Quy tắc: giảm partition (ví dụ trước khi ghi ra file) nên dùng coalesce; cần phân bố lại đều dữ liệu nên dùng repartition

Ví dụ: repartition, coalesce và Custom Partitioner

```
df_rdd = sc.textFile("big_file.csv", minPartitions=200)

# Giam xuong 10 partition truoc khi ghi file, tranh shuffle toan bo
ket_qua = df_rdd.filter(lambda l: l.strip() != "").coalesce(10)
ket_qua.saveAsTextFile("output/cleaned")

# Custom partitioner: phan vung theo domain de xu ly theo nhom
def phan_vung_theo_domain(key):
    return hash(key.split("@")[-1]) % 8

pairs = sc.parallelize([("a@vnu.edu.vn", 1), ("b@gmail.com", 2)])
pairs.partitionBy(8, phan_vung_theo_domain)
```

Vị trí của RDD trong hệ sinh thái Spark hiện đại

- Kể từ Spark 2.x, DataFrame/Dataset là API được khuyến nghị cho phần lớn công việc nhờ tối ưu hóa tự động (Catalyst — sẽ học ở phần sau)
- RDD vẫn là **nền tảng** bên dưới — mọi DataFrame cuối cùng đều được biên dịch thành các thao tác trên RDD
- RDD vẫn cần thiết khi xử lý dữ liệu **phi cấu trúc phức tạp** hoặc cần kiểm soát chi tiết từng bước biến đổi

Case study nhỏ: lọc & đếm log lỗi bằng RDD API

- Bài toán: từ log máy chủ, đếm số lượng lỗi HTTP 5xx theo từng endpoint
- `sc.textFile(...)` → `filter`(chỉ giữ dòng có mã lỗi 5xx) → `map`(trích endpoint) → `reduceByKey`(cộng dồn) → `collect`
- Đây chính là khuôn mẫu WordCount áp dụng vào một bài toán vận hành hệ thống thực tế

Câu hỏi kiểm tra nhanh #2

- Vì sao `reduceByKey` thường nhanh hơn `groupByKey` cho cùng một bài toán tổng hợp?
- Nếu một Executor bị crash giữa chừng, Spark khôi phục dữ liệu đã mất bằng cơ chế nào — có cần đọc lại toàn bộ dữ liệu gốc không?
- $\Rightarrow$ Thảo luận nhanh trước khi chuyển sang DataFrame & Spark SQL

Tổng kết: Kiến trúc Spark & RDD

Những điểm mấu chốt:

- Spark giữ dữ liệu trung gian trong RAM, dùng lineage thay vì replicate để chịu lỗi
- Transformation là lazy; Action mới kích hoạt thực thi toàn bộ DAG đã tích lũy
- Narrow dependency không cần shuffle; Wide dependency cần shuffle và tạo ranh giới Stage mới
- Broadcast Variable và Accumulator là hai công cụ chia sẻ trạng thái hiệu quả giữa Driver và Executor
- repartition() shuffle toàn bộ; coalesce() gộp partition không cần shuffle toàn bộ

Câu hỏi ôn tập

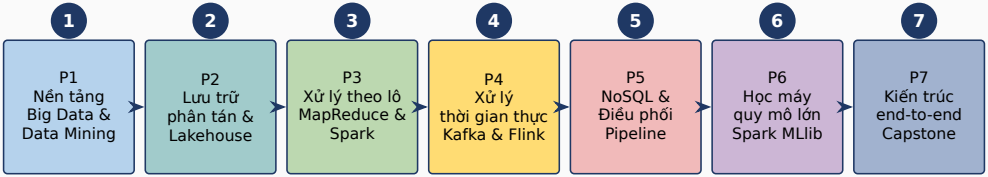
1. Vẽ lại DAG của đoạn code: textFile → map → filter → reduceByKey → collect, chỉ rõ ranh giới Stage
2. Vì sao Broadcast Variable giúp tránh được một phép shuffle tốn kém trong bài toán join bảng nhỏ–lớn?

Chuyển tiếp: từ RDD đến DataFrame

- RDD linh hoạt nhưng **không biết** cấu trúc dữ liệu bên trong — Spark không thể tối ưu truy vấn dựa trên schema
- Phần tiếp theo giới thiệu **DataFrame**: RDD có schema (tên cột, kiểu dữ liệu), cùng với **Spark SQL** và bộ máy tối ưu hóa Catalyst
- Đây chính là lớp API mà phần lớn công việc ETL/phân tích dữ liệu thực tế sử dụng hàng ngày

DATAFRAME, SPARK SQL & CATALYST OPTIMIZER

Từ dữ liệu có cấu trúc đến tối ưu hóa truy vấn tự động



Từ RDD đến DataFrame — vì sao cần schema

- DataFrame: một RDD có thêm **schema** (tên cột, kiểu dữ liệu) — giống một bảng trong CSDL quan hệ
- Nhờ biết schema, Spark có thể **tối ưu hóa tự động** (Catalyst) — điều RDD thuần không làm được vì Spark không hiểu cấu trúc bên trong
- Dataset (Scala/Java): DataFrame có kiểm tra kiểu dữ liệu tại thời điểm biên dịch (type-safe); PySpark chỉ có DataFrame

Tạo DataFrame từ CSV

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("Tuan4-DataFrame").getOrCreate()

df = spark.read.csv(
    "sales.csv",
    header=True,
    inferSchema=True,
)
df.printSchema()
df.show(5)
```

Tạo DataFrame từ JSON và Parquet

```
# JSON: moi dong la mot doi tuong JSON
df_json = spark.read.json("events.jsonl")

# Parquet: dinh dang cot, giu nguyen schema, doc nhanh hon CSV/JSON rat nhieu
df_parquet = spark.read.parquet("s3a://bucket/sales_parquet/")

df_parquet.select("category", "revenue").show(5)
```

Tạo DataFrame từ JDBC (kết nối CSDL quan hệ)

```
df_jdbc = (spark.read
    .format("jdbc")
    .option("url", "jdbc:postgresql://localhost:5432/shop")
    .option("dbtable", "orders")
    .option("user", "bigdata")
    .option("password", "******")
    .load())

df_jdbc.show(5)
```

Schema: inferSchema vs. khai báo StructType tường minh

- `inferSchema=True`: Spark tự đoán kiểu dữ liệu bằng cách quét qua dữ liệu — tiện nhưng **chậm hơn** và có thể đoán sai
- Khai báo **StructType** tường minh: nhanh hơn (không cần quét trước), đảm bảo đúng kiểu dữ liệu ngay từ đầu — nên dùng trong pipeline production

Ví dụ khai báo StructType tường minh

```
from pyspark.sql.types import (
    StructType, StructField, StringType, DoubleType, IntegerType
)

schema = StructType([
    StructField("category", StringType(), nullable=False),
    StructField("product",  StringType(), nullable=False),
    StructField("quantity", IntegerType(), nullable=True),
    StructField("revenue",  DoubleType(), nullable=True),
])

df = spark.read.csv("sales.csv", header=True, schema=schema)
```

DataFrame transformations cơ bản: select, filter, withColumn

```
from pyspark.sql import functions as F

ket_qua = (df
    .select("category", "product", "quantity", "revenue")
    .filter(df.quantity > 0)
    .withColumn("revenue_vnd", F.col("revenue") * 25400)
)
ket_qua.show(5)
```

groupBy + agg — ví dụ tổng hợp doanh thu theo danh mục

```
(df.filter(df.quantity > 0)
   .groupBy("category")
   .agg(F.sum("revenue").alias("total_revenue"),
        F.count("*").alias("num_orders"),
        F.avg("revenue").alias("avg_revenue"))
   .orderBy(F.desc("total_revenue"))
   .show())
```

Các loại Join trong DataFrame API

- **inner**: chỉ giữ dòng khớp ở cả hai bảng (mặc định)
- **left / right / outer**: giữ toàn bộ dòng của một hoặc cả hai bảng, điền `null` nếu không khớp
- **left_semi**: giữ dòng bên trái **có khớp** bên phải, nhưng chỉ lấy cột bên trái (giống lọc theo điều kiện tồn tại)
- **left_anti**: ngược lại — giữ dòng bên trái **không khớp** với bên phải (tìm phần tử thiếu)

Ví dụ các loại Join

```
don_hang = spark.read.csv("orders.csv", header=True, inferSchema=True)
khach_hang = spark.read.csv("customers.csv", header=True, inferSchema=True)

inner = don_hang.join(khach_hang, "customer_id", "inner")
left  = don_hang.join(khach_hang, "customer_id", "left")

# Tim khach hang CHUA TUNG dat don hang nao (left_anti)
chua_mua = khach_hang.join(don_hang, "customer_id", "left_anti")
chua_mua.show()
```

Spark SQL: viết truy vấn bằng cú pháp SQL quen thuộc

- Spark SQL cho phép truy vấn DataFrame bằng cú pháp SQL — rất hữu ích cho người đã quen SQL truyền thống (nhắc lại Phần 2: Schema-on-read)
- Đăng ký một DataFrame thành **temporary view** để có thể truy vấn bằng `spark.sql(...)`

Ví dụ: createOrReplaceTempView + spark.sql()

```
df.createOrReplaceTempView("sales")

ket_qua = spark.sql("""
    SELECT category,
           SUM(revenue) AS total_revenue,
           COUNT(*)     AS num_orders
    FROM sales
    WHERE quantity > 0
    GROUP BY category
    ORDER BY total_revenue DESC
""")
ket_qua.show()
```

DataFrame API vs. Spark SQL — cùng bài toán, hai cách viết

DataFrame API

- Phong cách lập trình hàm (method chaining)
- Kiểm tra cú pháp ngay lúc viết code (IDE hỗ trợ tốt)

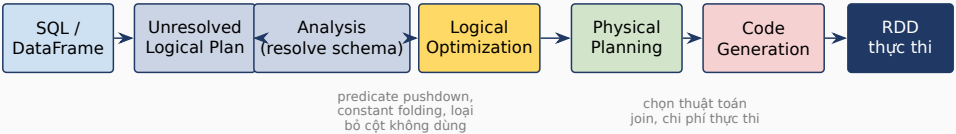
Spark SQL

- Cú pháp SQL quen thuộc với dân phân tích dữ liệu
- Cả hai đều được Catalyst biên dịch thành **cùng một** kế hoạch thực thi vật lý

Catalyst Optimizer là gì?

- Bộ máy tối ưu hóa truy vấn của Spark SQL, tự động biến đổi truy vấn (dù viết bằng DataFrame API hay SQL) thành kế hoạch thực thi hiệu quả nhất
- Nhờ Catalyst, DataFrame/Spark SQL thường nhanh hơn RDD thuần dù mã nguồn viết đơn giản hơn nhiều

4 giai đoạn của Catalyst Optimizer



Từ truy vấn SQL/DataFrame đến kế hoạch vật lý cuối cùng, qua Analysis, Logical Optimization, Physical Planning và Code Generation

Giai đoạn Analysis

- Giải quyết tên cột/bảng thành tham chiếu cụ thể ("resolve"), tra cứu schema thực tế
- Kiểm tra kiểu dữ liệu có hợp lệ không (ví dụ không thể cộng một cột kiểu String với Integer mà không ép kiểu)
- Nếu truy vấn tham chiếu một cột không tồn tại, lỗi được phát hiện **ngay ở giai đoạn này**, trước khi chạy thật

Giai đoạn Logical Optimization — ví dụ cụ thể

- **Predicate Pushdown**: đẩy điều kiện WHERE xuống **càng sớm càng tốt**, thậm chí xuống tận lúc đọc file Parquet (chỉ đọc đúng row-group cần thiết)
- **Constant Folding**: tính trước các biểu thức hằng số (ví dụ `1 + 1` $\rightarrow$ `2`) ngay khi lập kế hoạch, không tính lại mỗi dòng
- **Column Pruning**: chỉ đọc đúng những cột thực sự được dùng trong truy vấn, bỏ qua các cột còn lại (đặc biệt hiệu quả với định dạng cột như Parquet)

Ví dụ Predicate Pushdown thực tế (EXPLAIN)

```
df = spark.read.parquet("sales_parquet/")
ket_qua = df.filter(df.category == "Dien thoai").select("product", "revenue")

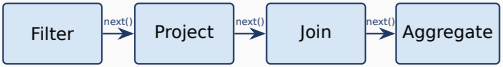
ket_qua.explain(True)
# ... PushedFilters: [IsNotNull(category), EqualTo(category,Dien thoai)] ...
# --> Dieu kien WHERE da duoc day xuong tan lop doc file Parquet,
#     khong phai doc het roi moi loc trong bo nho
```

Giai đoạn Physical Planning

- Từ một kế hoạch logic đã tối ưu, Catalyst sinh ra **nhiều kế hoạch vật lý** khả thi (ví dụ nhiều cách thực hiện join khác nhau)
- Dùng **mô hình chi phí (cost model)** dựa trên thống kê dữ liệu (kích thước bảng, số dòng...) để chọn kế hoạch **rẻ nhất**
- Đây chính là bước quyết định, ví dụ, có dùng Broadcast Hash Join hay Sort-Merge Join (sẽ xem chi tiết ở phần sau)

Tungsten: Whole-Stage Code Generation

Volcano model (thông dịch, gọi next() liên tục)



Mỗi bước gọi hàm ảo (virtual call) riêng — nhiều overhead

Whole-stage Code Generation (Tungsten)

```
for (row : input) { if (filter) { project(row); join(row); aggregate(row); } }
```

Toàn bộ pipeline biên dịch thành một vòng lặp Java bytecode duy nhất

Thay vì gọi hàm ảo (virtual call) cho từng toán tử riêng lẻ, Tungsten biên dịch cả một chuỗi toán tử thành một vòng lặp mã máy duy nhất

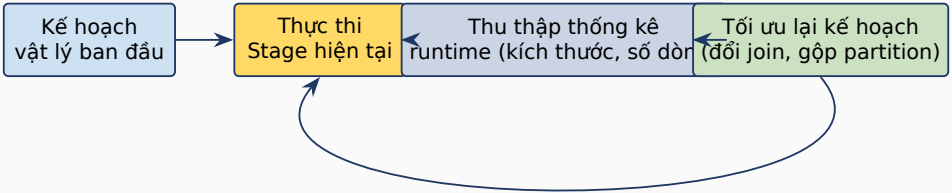
Vì sao Code Generation giúp Spark nhanh hơn?

- Mô hình cũ (Volcano): mỗi toán tử (Filter, Project, Join...) là một đối tượng riêng, gọi `next()` lẫn nhau — nhiều **virtual call**, khó tối ưu ở cấp CPU
- **Whole-Stage Code Generation**: Tungsten sinh mã Java bytecode hợp nhất nhiều toán tử thành **một vòng lặp duy nhất**, tận dụng tốt CPU cache và loại bỏ overhead gọi hàm
- Tungsten còn quản lý bộ nhớ **off-heap** (ngoài vùng nhớ do JVM Garbage Collector quản lý) để giảm chi phí GC với dữ liệu lớn

Adaptive Query Execution (AQE) là gì?

- Trước Spark 3.0: toàn bộ kế hoạch thực thi được quyết định **một lần, trước khi chạy**, dựa trên ước lượng (có thể sai lệch nhiều so với thực tế)
- AQE (mặc định bật từ Spark 3.x): Spark **thu thập số liệu thực tế** sau mỗi Stage và **tối ưu lại** phần kế hoạch còn lại dựa trên số liệu đó

Luồng hoạt động của AQE



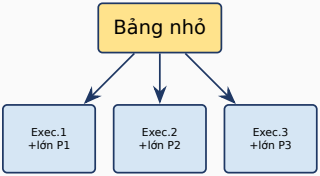
Sau mỗi Stage, Spark thu thập thống kê runtime và có thể thay đổi chiến lược cho các Stage tiếp theo

AQE trong thực tế — ba khả năng tự động điều chỉnh

- **Gộp shuffle partition nhỏ**: nếu số liệu thực tế cho thấy nhiều partition quá nhỏ, AQE tự gộp lại, tránh lãng phí overhead khởi động task
- **Đổi chiến lược Join**: nếu một bảng hóa ra nhỏ hơn dự kiến sau khi lọc, AQE có thể chuyển từ Sort-Merge Join sang Broadcast Hash Join giữa chừng
- **Xử lý Data Skew tự động**: phát hiện partition quá lớn bất thường và tự động chia nhỏ nó ra để cân bằng tải (sẽ bàn kỹ ở phần hiệu năng)

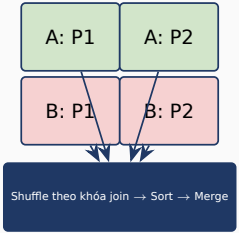
Broadcast Hash Join vs. Sort-Merge Join

Broadcast Hash Join



Không shuffle bảng lớn — join cục bộ

Sort-Merge Join



Cả hai bảng đều được shuffle theo khóa

Broadcast Hash Join tránh shuffle bảng lớn khi bảng còn lại đủ nhỏ; Sort-Merge Join shuffle cả hai bảng theo khóa join

So sánh chi tiết các chiến lược Join

Tiêu chí	Broadcast Hash Join	Sort-Merge Join
Điều kiện áp dụng	Một bảng đủ nhỏ để gửi toàn bộ đến mọi Executor	Cả hai bảng đều lớn
Shuffle	Không shuffle bảng lớn	Shuffle cả hai bảng theo khóa join
Tốc độ	Rất nhanh khi điều kiện thỏa mãn	Chậm hơn nhưng luôn áp dụng được

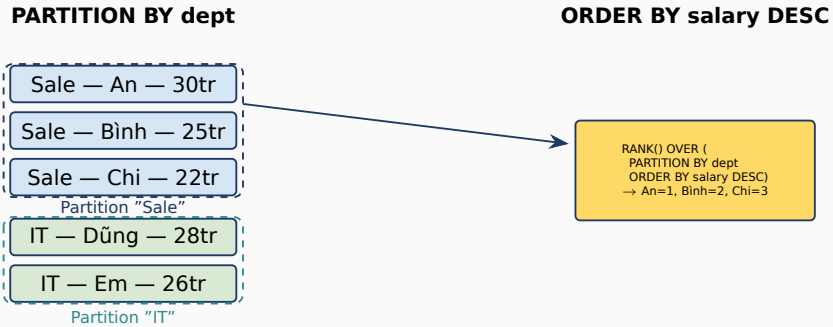
Broadcast Join Threshold

- Cấu hình `spark.sql.autoBroadcastJoinThreshold` (mặc định 10MB): bảng nhỏ hơn ngưỡng này sẽ tự động được broadcast
- Có thể ép broadcast tường minh bằng `F.broadcast(df_nho)` khi biết chắc bảng nhỏ nhưng Spark ước lượng kích thước chưa chính xác
- Đặt ngưỡng quá cao có thể khiến Spark broadcast nhầm một bảng thực ra khá lớn — gây tràn bộ nhớ Executor

Window Functions trong Spark SQL là gì?

- Cho phép tính toán **dựa trên một nhóm dòng liên quan** (window/frame) mà **không làm giảm số dòng kết quả** — khác với `GROUP BY`
- Cú pháp: `<hàm> OVER (PARTITION BY ... ORDER BY ...)`
- Ứng dụng phổ biến: xếp hạng (ranking), tính tổng lũy kế (running total), so sánh với dòng trước/sau (LAG/LEAD)

Minh họa PARTITION BY / ORDER BY



Dữ liệu được chia thành các partition (theo phòng ban), sắp xếp trong mỗi partition, rồi hàm window tính toán trên từng nhóm

Ví dụ Window Function: xếp hạng lương theo phòng ban

```
from pyspark.sql.window import Window
from pyspark.sql import functions as F

cua_so = Window.partitionBy("dept").orderBy(F.desc("salary"))

nhan_vien.withColumn("rank_trong_phong", F.rank().over(cua_so)) \
         .show()
# Sale - An   - 30tr -> rank 1
# Sale - Binh - 25tr -> rank 2
# IT   - Dung - 28tr -> rank 1  (rank tinh lai tu dau moi partition)
```

Ví dụ Window Function: tổng lũy kế doanh thu theo ngày

```
cua_so_luy_ke = (Window.partitionBy("category")
                        .orderBy("order_date")
                        .rowsBetween(Window.unboundedPreceding, Window.currentRow))

doanh_thu.withColumn(
    "luy_ke", F.sum("revenue").over(cua_so_luy_ke)
).show()
```

User Defined Function (UDF) — khi nào cần?

- Dùng khi logic xử lý **quá phức tạp hoặc quá đặc thù** để biểu diễn bằng các hàm built-in có sẵn (`F.col`, `F.when`...)
- Ví dụ: áp dụng một mô hình phân loại văn bản đơn giản, hoặc một quy tắc nghiệp vụ phức tạp không có sẵn hàm SQL tương ứng

Ví dụ viết và đăng ký UDF

```
from pyspark.sql.types import StringType

def phan_loai_gia_tri(revenue):
    if revenue is None:
        return "khong_xac_dinh"
    elif revenue < 100:
        return "thap"
    elif revenue < 1000:
        return "trung_binh"
    else:
        return "cao"

phan_loai_udf = F.udf(phan_loai_gia_tri, StringType())
df.withColumn("nhom_gia_tri", phan_loai_udf(df.revenue)).show()
```

Cảnh báo hiệu năng: UDF thông thường và Catalyst

- UDF thông thường là một **hộp đen** đối với Catalyst — không thể tối ưu hóa bên trong nó (không predicate pushdown, không code generation)
- Với Python UDF, dữ liệu còn phải **tuần tự hóa qua lại** giữa JVM và tiến trình Python cho **từng dòng** — overhead rất lớn
- Nguyên tắc: luôn ưu tiên hàm built-in trước; chỉ dùng UDF khi thực sự không có lựa chọn khác

Ví dụ Pandas UDF (vectorized) — nhanh hơn UDF thường

```
import pandas as pd
from pyspark.sql.functions import pandas_udf

@pandas_udf(StringType())
def phan_loai_vectorized(revenue: pd.Series) -> pd.Series:
    return pd.cut(
        revenue, bins=[-1, 100, 1000, float("inf")],
        labels=["thap", "trung_binh", "cao"]
    ).astype(str)

df.withColumn("nhom_gia_tri", phan_loai_vectorized(df.revenue)).show()
# Xu ly theo tung LO (batch) bang Pandas/Arrow thay vi tung dong -> nhanh hon nhieu
```

Tổng hợp các cấu hình Spark SQL thường dùng

Cấu hình	Ý nghĩa
spark.sql.shuffle.partitions	Số partition mặc định sau mỗi phép shuffle (mặc định 200)
spark.sql.autoBroadcastJoinThreshold	Ngưỡng kích thước bảng để tự động broadcast (mặc định 10MB)
spark.sql.adaptive.enabled	Bật/tắt Adaptive Query Execution (mặc định bật từ Spark 3.x)
spark.serializer	Chọn Kryo thay vì Java Serialization mặc định để tăng tốc

Câu hỏi kiểm tra nhanh #3

- Predicate pushdown và column pruning giúp tiết kiệm chi phí đọc dữ liệu như thế nào khi nguồn dữ liệu là Parquet?
- Vì sao AQE cần chạy **sau khi** một Stage đã thực thi xong, thay vì quyết định hết mọi thứ ngay từ đầu?
- $\Rightarrow$ Thảo luận nhanh trước khi chuyển sang phần hiệu năng & case study

Tổng kết: DataFrame, Spark SQL, Catalyst — Những điểm mấu chốt

- DataFrame = RDD + schema, cho phép Catalyst tối ưu hóa tự động mà RDD thuần không có được
- Catalyst gồm 4 giai đoạn: Analysis, Logical Optimization, Physical Planning, Code Generation
- Tungsten sinh mã máy hợp nhất (whole-stage codegen) và quản lý bộ nhớ off-heap để tăng tốc thực thi
- AQE tối ưu lại kế hoạch dựa trên số liệu runtime thực tế, kể cả đổi chiến lược Join giữa chừng
- Window Function tính toán theo nhóm mà không giảm số dòng; UDF linh hoạt nhưng nên hạn chế vì cản trở tối ưu hóa

Tổng kết: DataFrame, Spark SQL, Catalyst — Câu hỏi ôn tập

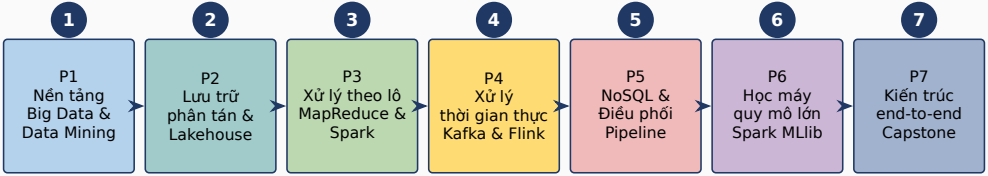
- Vì sao DataFrame/Spark SQL thường nhanh hơn RDD thuần cho cùng một bài toán?
- Trình bày lại sự khác biệt giữa Broadcast Hash Join và Sort-Merge Join, và điều kiện Spark chọn mỗi loại
- Vì sao nên ưu tiên Pandas UDF (vectorized) hơn UDF Python thông thường khi bắt buộc phải dùng UDF?

Chuyển tiếp: hiệu năng thực tế và gỡ lỗi Spark

- Đã hiểu Spark hoạt động **đúng** như thế nào (kiến trúc, RDD, DataFrame, Catalyst)
- Phần tiếp theo trả lời câu hỏi thực dụng hơn: làm sao biết một job Spark đang chạy **chậm vì đâu**, và cách khắc phục cụ thể

HIỆU NĂNG, GỠ LỖI & NGHIÊN CỨU TÌNH HUỐNG

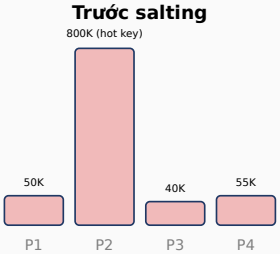
Từ lý thuyết đến vận hành Spark trong thực tế



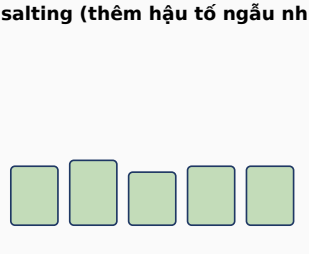
Data Skew — vấn đề hiệu năng phổ biến nhất

- Xảy ra khi dữ liệu **phân bố không đều** giữa các partition — một vài khóa (hot key) chiếm phần lớn dữ liệu
- Hậu quả: một Task xử lý partition đó chạy **rất lâu**, trong khi các Task khác đã xong từ sớm — toàn bộ Stage phải chờ Task chậm nhất
- Ví dụ thực tế: bảng giao dịch có cột `country`, phần lớn giao dịch đến từ Việt Nam — groupBy theo country tạo ra một partition khổng lồ

Data Skew: trước và sau khi Salting



Một Task xử lý P2 chạy chậm hơn hẳn — kéo dài toàn bộ Stage



Khóa hot được chia thành nhiều khóa phụ — tải phân bố đều hơn

Kỹ thuật salting chia nhỏ một khóa hot thành nhiều khóa phụ ngẫu nhiên, giúp tải được phân bố đều hơn giữa các partition

Kỹ thuật Salting để giảm Data Skew

```
from pyspark.sql import functions as F

# Them hau to ngau nhien (0-9) vao khoa hot truoc khi groupBy
df_salted = df.withColumn(
    "key_salted",
    F.concat(df.country, F.lit("_"), (F.rand() * 10).cast("int"))
)

tam = df_salted.groupBy("key_salted").agg(F.sum("amount").alias("sub_total"))

# Gop lai ket qua theo khoa goc (bo hau to salt)
ket_qua = (tam
    .withColumn("country", F.split(tam.key_salted, "_")[0])
    .groupBy("country")
    .agg(F.sum("sub_total").alias("total")))
```

Các nguyên tắc tối ưu hiệu năng Spark tổng hợp

- Cache lại các DataFrame/RDD được **dùng lại nhiều lần**, nhưng không cache tràn lan (tốn bộ nhớ Executor)
- Ưu tiên hàm built-in và Pandas UDF thay vì UDF Python thông thường
- Chủ động dùng `F.broadcast()` khi biết chắc một bảng nhỏ nhưng Spark ước lượng chưa chính xác
- Bật AQE (mặc định từ Spark 3.x) để tận dụng tối ưu hóa runtime tự động

Kích thước Partition hợp lý — quy tắc kinh nghiệm

- Quy tắc phổ biến: mỗi partition nên nằm trong khoảng **128MB–1GB** dữ liệu sau khi đọc vào bộ nhớ
- **Quá nhiều partition nhỏ**: overhead khởi động task lớn hơn công việc thực sự cần làm
- **Quá ít partition lớn**: không tận dụng hết số lõi CPU của cluster, dễ tràn bộ nhớ Executor

Kiểm tra và điều chỉnh số lượng Partition

```
print("So partition hien tai:", df.rdd.getNumPartitions())

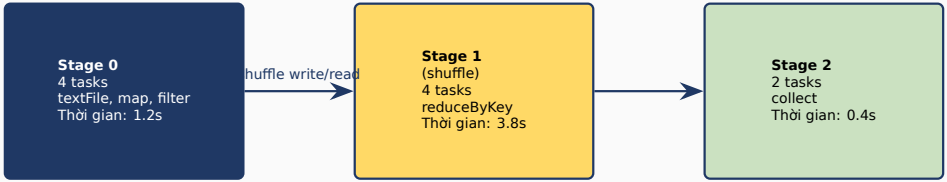
# Dieu chinh so shuffle partition mac dinh (mac dinh la 200)
spark.conf.set("spark.sql.shuffle.partitions", 50)

# Kiem tra kich thuoc du lieu tung partition (de phat hien skew)
from pyspark.sql.functions import spark_partition_id
df.withColumn("pid", spark_partition_id()) \
  .groupBy("pid").count().orderBy(F.desc("count")).show()
```

Đọc Spark UI để chẩn đoán hiệu năng

- Mỗi SparkSession tự khởi chạy một Web UI (mặc định `http://localhost:4040`)
- Tab **Jobs**: danh sách các Action đã kích hoạt và thời gian chạy tổng thể
- Tab **Stages**: chi tiết từng Stage, số Task, và **phân bố thời gian giữa các Task** — nơi phát hiện data skew rõ nhất

Ví dụ minh họa: đọc DAG trên Spark UI



Stage có shuffle (Stage 1) thường chiếm phần lớn thời gian thực thi — đây là dấu hiệu đầu tiên cần chú ý khi tối ưu

Các dấu hiệu cảnh báo thường gặp trên Spark UI

- **Một Task chạy lâu bất thường** so với các Task cùng Stage → nghi ngờ data skew
- **Nhiều "Spill (Memory)" hoặc "Spill (Disk)"**→ Executor thiếu RAM, dữ liệu phải tràn ra đĩa
- **Thời gian GC (Garbage Collection) cao** → quá nhiều object nhỏ, cân nhắc serialized storage level hoặc tăng bộ nhớ Executor

Lỗi thường gặp: OutOfMemoryError

- Nguyên nhân phổ biến: `collect()` một DataFrame quá lớn về Driver, hoặc broadcast nhầm một bảng lớn
- Cách xử lý: tăng bộ nhớ Executor/Driver (`spark.executor.memory`), tránh `collect()` toàn bộ dữ liệu lớn, kiểm tra lại data skew

Lỗi thường gặp: Serialization error

- Xảy ra khi Spark không tuần tự hóa (serialize) được một object để gửi qua mạng giữa Driver và Executor
- Mặc định Spark dùng Java Serialization (chậm); nên chuyển sang **Kryo Serializer** (`spark.serializer`) để nhanh và gọn hơn
- Lỗi thường gặp khi vô tình tham chiếu một biến/kết nối (ví dụ kết nối CSDL) không serialize được từ bên trong một hàm map

Checklist tối ưu hiệu năng Spark trước khi đưa vào production

- ☐ Đã kiểm tra phân bố kích thước partition, không còn data skew nghiêm trọng
- ☐ Đã cache đúng những DataFrame/RDD dùng lại nhiều lần, và unpersist khi không còn cần
- ☐ Đã broadcast tường minh các bảng nhỏ tham gia join
- ☐ Đã bật AQE và kiểm tra `spark.sql.shuffle.partitions` phù hợp quy mô dữ liệu
- ☐ Đã xem qua Spark UI ít nhất một lần để xác nhận không có Stage bất thường

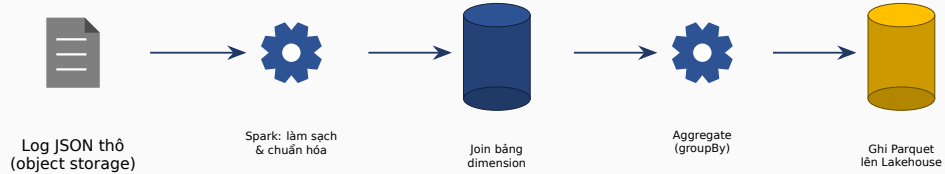
So sánh mở rộng: MapReduce vs. Spark

Tiêu chí	MapReduce	Spark
Lưu trung gian	Đĩa (disk)	Bộ nhớ (RAM), có thể tràn ra đĩa
Tốc độ (workload lặp)	Chậm	Nhanh hơn nhiều lần
API	Map/Reduce (Java)	RDD, DataFrame, SQL (đa ngôn ngữ)
Tối ưu hóa truy vấn	Không có	Catalyst + Tungsten + AQE
Phù hợp	ETL đơn giản, một lượt	ETL, ML, truy vấn tương tác, streaming

Benchmark tham khảo: Spark nhanh hơn MapReduce bao nhiêu?

Nguồn	Kết quả nổi bật
Daytona GraySort 2014 (Databricks)	Sắp xếp 100TB dữ liệu: Spark dùng 207 máy trong 23 phút, trong khi kỷ lục Hadoop MapReduce trước đó dùng 2100 máy trong 72 phút
AMPLab (khảo sát nội bộ)	Với thuật toán lặp (ví dụ Logistic Regression), Spark nhanh hơn MapReduce tới 10–100 lần nhờ giữ dữ liệu trong RAM giữa các vòng lặp

Case study: Pipeline ETL end-to-end



Log JSON thô → làm sạch/chuẩn hóa → join bảng dimension → tổng hợp → ghi Parquet lên Lakehouse

Case study ETL — bước 1: đọc & làm sạch log JSON

```
raw = spark.read.json("s3a://logs/2026-08-*/*.jsonl")

sach = (raw
    .filter(raw.user_id.isNotNull())
    .withColumn("event_time", F.to_timestamp("ts"))
    .dropDuplicates(["event_id"]))

sach.cache()   # se dung lai o buoc join va aggregate ben duoi
```

Case study ETL — bước 2: join dimension & tổng hợp

```
dim_product = F.broadcast(
    spark.read.parquet("s3a://dim/products/")   # bang nho -> broadcast join
)

lam_giau = sach.join(dim_product, "product_id", "left")

tong_hop = (lam_giau
    .groupBy("category", F.date_trunc("day", "event_time").alias("ngay"))
    .agg(F.sum("amount").alias("total_revenue"),
         F.countDistinct("user_id").alias("unique_users")))
```

Case study ETL — bước 3: ghi kết quả ra Parquet/Lakehouse

```
(tong_hop
    .repartition(10)                        # kiem soat so file dau ra
    .write
    .format("delta")                        # ghi dang bang Lakehouse (Phan 2)
    .mode("overwrite")
    .partitionBy("ngay")
    .save("s3a://lakehouse/daily_revenue/"))
```

Case study ETL — bài học rút ra

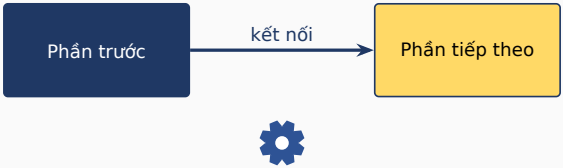
- `cache()` DataFrame `sach` vì nó được dùng lại ở cả bước join và tổng hợp
- Chủ động `broadcast` bảng dimension nhỏ để tránh shuffle bảng log lớn khi join
- Kiểm soát số lượng file đầu ra bằng `repartition` trước khi ghi, tránh tạo ra hàng nghìn file nhỏ trên Lakehouse
- Ghi dạng bảng Lakehouse (Delta/Iceberg — Phần 2) thay vì file Parquet thô để có ACID và time travel

Vị trí của Phần 3 (mở rộng) trong Roadmap

- Phần 3 là **công cụ xử lý trung tâm** — Spark sẽ xuất hiện lại ở hầu hết các phần sau: Streaming (P4), MLlib (P6), và là thành phần chính trong kiến trúc end-to-end (P7)
- Các kỹ năng tối ưu hiệu năng (skew, partition, Spark UI) học ở đây áp dụng trực tiếp cho mọi phần sau



Cầu nối sang Phần 4



*MapReduce và Spark (ở chế độ mặc định) xử lý dữ liệu **đã có sẵn** theo lô. Nhưng nhiều bài toán (Phần Ngữ cảnh & Ứng dụng) cần xử lý ngay khi dữ liệu phát sinh:*

- Gian lận thẻ tín dụng, cảnh báo IoT không thể chờ đến job batch tiếp theo
- → Phần 4 giới thiệu Kafka và Spark Structured Streaming/Flink cho xử lý thời gian thực, tái sử dụng toàn bộ DataFrame API vừa học

Tổng kết toàn bộ Phần 3 (bản mở rộng) — Những điểm mấu chốt

- MapReduce: mô hình Map–Shuffle–Reduce đơn giản, chịu lỗi tốt, nhưng chậm với workload lặp vì ghi đĩa liên tục
- Spark: giữ dữ liệu trong RAM, dùng lineage để chịu lỗi, nhanh hơn MapReduce nhiều lần với workload lặp/tương tác
- DataFrame + Catalyst + Tungsten + AQE là bộ ba giúp Spark vừa dễ viết code vừa tối ưu hóa tự động ở tầng thực thi
- Data Skew là nguyên nhân hiệu năng kém phổ biến nhất trong thực tế; Spark UI là công cụ chẩn đoán quan trọng nhất
- Case study ETL end-to-end minh họa cách các khái niệm (cache, broadcast join, partition) kết hợp với nhau trong một pipeline thực tế

Tổng kết toàn bộ Phần 3 (bản mở rộng) — Câu hỏi ôn tập

- So sánh chi tiết cơ chế chịu lỗi của MapReduce (chạy lại task) và Spark (tính lại theo lineage)
- Trình bày các bước Catalyst xử lý một truy vấn Spark SQL, từ lúc viết đến lúc thực thi
- Nếu một Job Spark chạy chậm bất thường, bạn sẽ kiểm tra những gì trên Spark UI theo thứ tự nào?